

MANDATE RULE No. 0

Mandate Ledger

An AI agent that can't write itself a blank check.



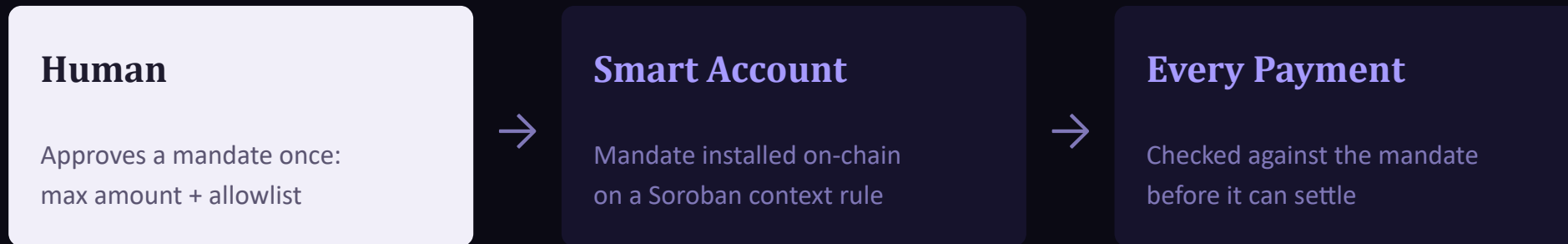
Spending mandates enforced on-chain · Soroban · Stellar testnet

THE PROBLEM

Autonomous agents need to pay for things. That's exactly the problem.

- 1 **An agent that can move money is useful** — paying vendors, topping up services, settling invoices without a human in the loop.
- 2 **But “trust the agent's own code” doesn't scale** — a bug, a bad prompt, or a compromised key turns into an unbounded blank check.
- 3 **Enforcement needs to live somewhere the agent can't touch** — not in the agent's logic, not in a UI warning, not in a policy nobody checks.

One human approval. Enforced on every decision after.



Enforcement lives in the blockchain — not in the agent's own code.

One rule at the center. Two contracts serve it.

THE RULE

mandate-policy

Max amount per transaction + recipient allowlist. Panics — and reverts the transaction — on any violation. This is the contract every payment is actually judged against.

100 XLM

MAX / TX

2

ALLOWLISTED PAYEES

smart-account

The agent's on-chain account (OpenZeppelin stellar-accounts). Every call routes through its authorization check first.

ed25519-verifier

Confirms the request really came from the agent's own bound key — before the mandate is even checked.

The mandate used in this demo

Filed once, by a human, at deployment. Never touched again.

100 XLM

MAX PER TRANSACTION

2

ALLOWLISTED PAYEES
cafe_supplier · cloud_provider

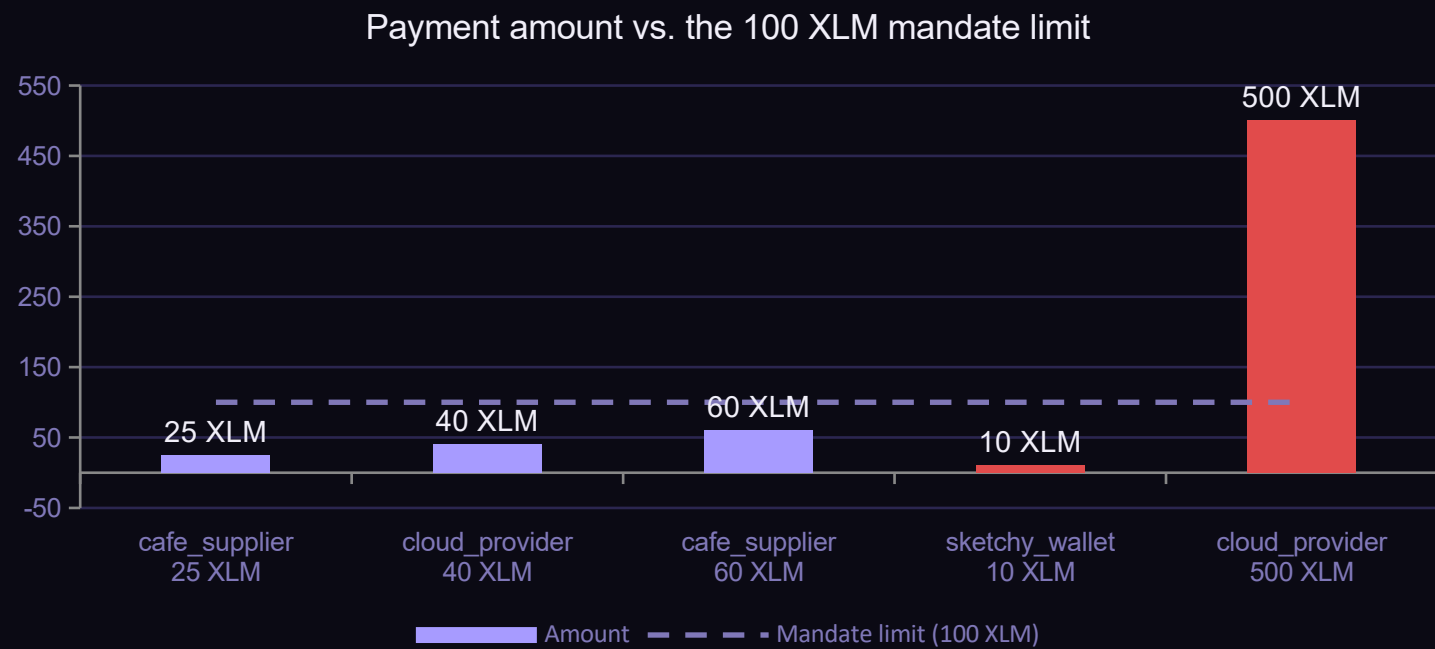
never

MANDATE EXPIRES

Approved once by a human — enforced by contract code from then on.

LIVE DEMO — RIGHT NOW

Five autonomous agent decisions, submitted live



OK

OK

OK

NO

NO

3 settled · 2 rejected on-chain · 0 silently skipped

Mandate Ledger

Soroban smart account · OpenZeppelin stellar-accounts

Stellar Testnet

Local control panel — secrets stay server-side in scripts/.env, never sent to this page. Runs against Stellar testnet only.

How this works +

ACCOUNT BALANCE

53 XLM

Held by the smart account

LIMIT PER TRANSACTION

100 XLM

Mandate v1 — adjustable by admin

ALLOWED PAYEES

2

cafe_supplier

cloud_provider

SETTLED THIS SESSION

0

Confirmed on testnet

REJECTED THIS SESSION

0

Blocked by the mandate

Every settled payment is independently verifiable

Real transaction. Real ledger. Real timestamp.

Anyone can paste the transaction hash into stellar.expert — no need to take our word for it.

- Confirmed on Soroban RPC (getTransaction: SUCCESS)
- Confirmed independently on Horizon testnet
- Ed25519 signature verified on-chain before the mandate ran

The screenshot shows the stellar.expert interface for a specific transaction. The header includes the site logo, navigation links (NETWORK STATS, ASSETS, MARKETS, SERVICES, BLOG), and a dropdown for the network (testnet). The main content area displays the transaction hash and a summary of its details. The summary includes the status (Successful), ledger (4679807), source account (GDW726...AW4KGE), sequence number (19930890226368522), transaction size (1,736 bytes), and the time it was processed (2026-09-14 21:50:22 UTC). It also shows the maximum fee (0.0040327 XLM) and the fee charged (0.0025102 XLM). Below the summary, the preconditions are listed, including the valid before time (2026-09-14 21:51:16). The transaction details section shows that the source account invoked a contract (CDLZ...CYSC) to transfer 250000000 i128 to the destination account (GBU4...D5L2). The signatures section shows the Ed25519 signature for the source account (GDW726...AW4KGE).

stellar.expert NETWORK STATS ASSETS MARKETS SERVICES BLOG Network testnet

Transaction df78754b78bdfabac6af0d11fc9014517f831f4db6207e0e461899ef7d5fe386

Summary

Status: Successful
Ledger: 4679807
Source Account: GDW726...AW4KGE
Sequence Number: 19930890226368522
Transaction size: 1,736 bytes
Preconditions
Valid before: 2026-09-14 21:51:16

Processed: 2026-09-14 21:50:22 UTC
Max Fee: 0.0040327 XLM
Fee Charged: 0.0025102 XLM (0.0020231 non-refundable + 0.0004771 refundable + 0.00001 min op fee)

GDW7...4KGE invoked contract CDLZ...CYSC transfer(CAGA...J03F, GBU4...D5L2, 250000000 i128)

Signatures

GDW726...AW4KGE: JBVLoA+NhueWbpXtExpKZyQtTZI+4zv8sLLT/yhV7fQ8UuixTQ65NHqaQ7bo8hBtgW3SFCcQhkgV/6J+mdCw==

2026 © StellarExpert v1.0.1
[Open API docs](#) [Report a bug](#) [Dark theme](#)
[Terms of use](#) [Privacy policy](#) [Status](#)

Two independent gates — both real

GATE 1

Identity

The ed25519-verifier checks the agent's real signature. Both rejected calls passed this — the agent proved exactly who it was.



GATE 2

Policy

The mandate-policy then checks amount + allowlist. Both calls were rejected here — for cause, with a specific error code.

Signature valid both times. The agent proved who it was — the mandate still said no.



One mandate. Approved once.

Enforced on every decision the agent makes afterward.

No way for the agent — or a bug in its own code — to bypass it.